IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2026, No. 2, pp. 28–50. DOI:10.46586/tches.v2026.i2.28-50

Racing against the clock: side-channel leakage from crossing asynchronous clock domains

Martin Thompson[1,2] and David Oswald[1]

[1] Durham University, Durham, UK, martin.j.thompson@durham.ac.uk, david.f.oswald@durham.ac.uk
[2] ZF Automotive UK Ltd, Solihull, UK, martin.j.thompson@zf.com

Abstract. Current physical side-channel attacks are based on direct power or electro-magnetic measurements, or indirect quantities such as precise measurements of clock jitter. On the other hand, timing side-channel attacks exfiltrate secrets by observing secret-dependent variations in runtime caused by branches or micro-architectural effects, *e.g.* due to caching. However, little attention has been paid to the fact that power leakage might also manifest architecturally, for example through variations in cycle counts of clock domain crossings, due to the supply-voltage-dependence of the Voltage-Controlled Oscillators within the on-chip Phase-Locked Loop blocks. In this paper, we show that memory and peripherals that operate in a different clock domain to a target processor can be used to indirectly measure power consumption. This allows power side-channel attacks to be performed from software, without privileged access to direct power measurements and using only a cycle counter.
We first demonstrate that in a (noisy) x86 environment, it is possible to differentiate between instructions (and instruction operands) based on the variation of cycle counts relative to wall-clock time. For an embedded environment (using an NXP i.MX RT1160-family System-on-Chip (SoC)), we similarly show surprisingly large variations in cycle count correlated with the Hamming weight of data. Based on these observations, we hypothesise a leakage mechanism and present evidence from simulation and a Field Programmable Gate Array-based experiment. We then construct a timing-independent covert channel based on this direct proxy for power consumption on the SoC. Going further, we show that it is possible to recover the private exponent from an RSA modular exponentiation as well as an AES key across a Real-time Operating System thread boundary.

Keywords: side-channel analysis · clocks · timing leakage · software-based side channels

1 Introduction

Side-channel analysis based on physical measurements of *e.g.* power consumption and electro-magnetic (EM) emanation has been widely explored in the literature [GMO01; Goo+11; VMC19; MOP10; Lon+15; Kim+19; SM22; Udv+21]. Building on this, in recent years, software-based methods to capture power side-channel leakage have emerged. For example, Lipp et al. demonstrated that low-resolution on-chip power measurements can be exploited to leak secrets across the Intel Software Guard Extensions (SGX) security barrier [Lip+21]. Gnad, Krautter, and Tahoori and O'Flynn and Dewar showed that on-chip peripherals such as analogue-to-digital converters leak significant information about power consumption, which can be exploited across security boundaries such as Trustzone [GKT19; OD19]. Other peripherals that provide indirect measurement of power consumption include delay-lines [Gra+21].

Licensed under Creative Commons License CC-BY 4.0.
Received: 2025-10-15 Accepted: 2025-12-15 Published: 2026-04-23

Further, Schoos et al., O'Flynn and Ayoub et al. demonstrated that precise measurement of clock variation (visible as phase-modulation or 'jitter') can also be used to infer power consumption [Sch+23; OFl24; Ayo+24]. These works use additional peripherals or lab equipment, connected to the target device, in order to make precise measurements of jitter. Considering workload-dependent clocking, or Dynamic Voltage and Frequency Scaling (DVFS), Wang et al. developed an attack that leaks data over longer timescales due to the heating effect on the processor [Wan+22].

While variations in execution time—*e.g.* due to branches, memory access patterns and cache activity—has been extensively studied as a source of leakage [Ber05; BB03], according attacks can be mitigated through "constant-time" software development patterns.

More precisely, we focus on the effect of *power consumption on execution time* in the absence of any dynamic clocking (*i.e.* DVFS) and assuming a target implementation free of (cache)timing leakage. We show that it is possible to measure leakage-in-time related to *short-term power consumption*, allowing us to apply simple clock-cycle timing measurements through the entire literature of power-based side-channels. To our knowledge, we are the first to explore and demonstrate this effect on short time scales.

1.1 Voltage-Dependent Clock Variations

In modern processors and system-on-chips, a Phase-Locked Loop (PLL) is often used to provide the core frequency (at a non-integer multiple of the source rate). PLLs consist of a control-loop, based on a Phase-Frequency Detector (PFD) (and a frequency divider, if a multiplication of the reference frequency is required). The voltage output of these circuits is filtered and fed to the control input of a Voltage-Controlled Oscillator (VCO) (see Figure 1). As the supply voltage V_{supply} varies, each block that is connected to it will undergo changes in electrical response [Abr02]. The VCO is the most sensitive item, resulting in direct, small changes in the output frequency. Every PLL has a parameter known as 'supply voltage pushing' which describes the transfer function from changes in supply voltage to changes in VCO frequency [Ban17].

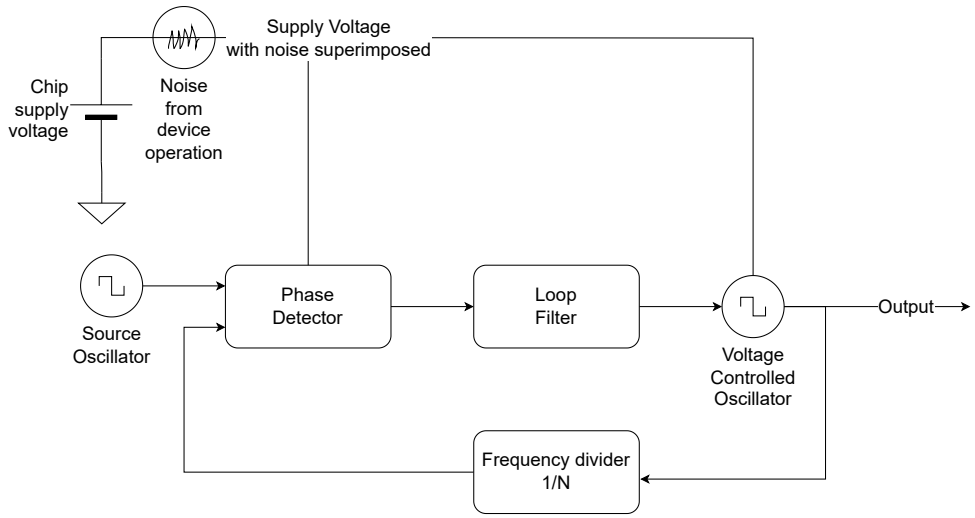


Figure 1: The basic structure of a phase-locked-loop, showing how the supply voltage can influence behaviour

1.2 Related Work

The minute changes in timing between different operations, based on the information provided to a system by an attacker, have been exploited many times. Private keys can be extracted from an undefended asymmetric cryptosystem based solely on the time it takes to perform an operation [Koc96]. Symmetric cryptosystems, particularly AES, are equally vulnerable to differences in timing for different attacker-provided inputs [Kel+98; Per05; Ber05; BM06]. This is due to software implementations of AES depending on one or more look-up tables (either the SBox or T-table) which are stored in memory which the processor accesses via its cache. More recent work goes beyond caches into deeper micro-architectural features [Lip+18; Koc+19], exploiting the fact that modern processor architectures speculatively execute instructions, which leaks information allowing an attacker to infer secrets.

Power analysis attacks use variations of the power consumption over time while a target device performs operations involving a secret. Current flow can be measured directly (by measuring the voltage drop across a series resistor [Ohm27]) or indirectly (by measuring the changes in electro-magnetic field [Max73]). Modern processors provide the ability to measure the short-term power consumption on-chip and make this measurement available to software [Lip+21]. Other mechanisms which can be used to infer power consumption include measurements of jitter on Serialiser/Deserialiser (SerDes) blocks [Sch+23], phase-modulation of the electro-magnetic emissions of a device [Ayo+24] or on the debugger pins [OFl24]. Logic in Field Programmable Gate Arrays (FPGAs) can provide measurements of delay (through frequency variations) that are proportional to power consumption [Udu+21; FBL24].

Once measurements of power are available, Simple Power Analysis (SPA) [KJJ99] directly interprets the power trace of the device under attack to deduce information about the secrets being used. Differential Power Analysis (DPA) [KJJ99] separates the power traces according to some prediction of the expected power consumption of the device using a single bit of the postulated secret conditions, and compares the average power consumptions of the two sets. When the guess of the secret is correct, there will be clear differences between the two sets. Absolute Sum Differential Power Analysis (AS-DPA) [Dog+11] develops the DPA concept further by combining the results of DPA on each bit through the sum of the absolute differences of the means. This combines the information available in a form which is robust to inaccurate modelling (as there is no explicit model invoked). Correlation Power Analysis (CPA) [BCO04] makes predictions of hypothetical power consumption, usually according to a Hamming Weight (HW) or Hamming-distance (HD) model, based on the attacker's guesses as to the secret information. A statistical correlation is performed between these predictions and the measured power traces. Those with the highest correlation correspond to the most likely guesses of the secret. Linear Regression Analysis (LRA) [SLP05] does not assume a specific leakage model for the power, but builds a model during a training phase, and uses this model as a distinguisher during the attack phase. The use of Mutual Information Analysis (MIA) to combine leakage across multiple bits has been evaluated [Gie+08]. This technique does not require a training phase, nor make any assumptions about the leakage-model to be used or the distribution of the noise, and has been shown to be effective, especially in the case of targets designed to resist side-channel analysis through physical implementation countermeasures (in this case the use of dual-rail precharge logic).

1.3 Contributions

Continuing in the line of work that measures power leakage through indirect and software-exposed side channels, in this paper we demonstrate that:

- variations in power consumption produce measurable variations in short-term clock frequency of a processor, and that these variations differ between clock domains;
- these small variations can be measured through processor cycle counters, and amplified in the presence of an asynchronously-clocked Random-Access Memory (RAM) block or other asynchronously-clocked peripherals;
- it is possible to exploit these timing changes to construct a covert channel across process boundaries;
- indirect cycle-counter measurements can be used to extract AES and RSA keys using standard power analysis techniques from software, even in the absence of data caching.

Responsible Disclosure We disclosed our findings regarding the leakage of the i.MX RT1166 to the vendor NXP on 11 November 2024. NXP acknowledged and reproduced the leakage, and our findings were discussed in several follow-up calls. We further provided NXP with a copy of the submitted paper. NXP have provided the following comment: *"To ensure authenticated code execution and protect sensitive data, NXP recommends enabling Secure Boot on the i.MX RT1160. Secure Boot establishes a hardware root of trust that validates firmware signatures at startup, preventing unauthorized or tampered code from executing. Additionally, leverage the Cryptographic Acceleration and Assurance Module (CAAM) for supported cryptographic operations, including AES, to keep key material inside secure hardware, reducing exposure and improving resistance to side-channel attacks. Together, Secure Boot and CAAM deliver a robust security foundation by guaranteeing trusted code and safeguarding cryptographic processes throughout the system lifecycle. "* For x86 systems, we did not find directly exploitable leakage that would warrant responsible disclosure at this point.

Data Availability The code and data underlying this paper is available at: https://doi.org/10.5281/zenodo.18146389

Outline The remainder of this paper is structured as follows: we begin in Section 2 with our initial investigations into this topic (using an x86 Linux environment and an NXP i.MX RT1166[1] Arm-based embedded System-on-Chip (SoC)), and the results which motivated our further activity. In Section 3 we explain the conjectured mechanism behind the leakage supported by evidence from a VHSIC (Very High Speed Integrated Circuit) Hardware Description Language (VHDL)-based simulation environment. Following this, in Section 4, we show an FPGA-based target to confirm the simulation results in real hardware while retaining strong control over the structure of the experimental platform. Returning to the i.MX RT1166 SoC, in Section 5 we demonstrate the use of a DMA channel (reading a timer) to provide information on the variation of power over time, akin to traditional power traces. From all these results, in Section 6 we demonstrate proof-of-concept attacks in real-world representative circumstances using the i.MX RT1166 SoC. We conclude in Section 7 with a discussion of our results and the implications for system design, including potential countermeasures.

[1] https://www.nxp.com/products/i.MX-RT1160

2 Initial Exploration

We sought to establish evidence in support of the hypothesis that the clocks available in a processing system will vary differently with power consumption. To this end, we captured clock measurements and the known power measurements available on Intel x86 processors using the Running Average Power Limit (RAPL) interface under various conditions of varying power. We performed these measurements on a PC-based platform (Section 2.1), and later moved to an embedded i.MX RT1166 SoC (Section 2.2).

2.1 Comparison of Clock Variations to Software Measurements on x86

Our first experiments used simple Python code to randomly execute one of two workloads: a high power-consumption workload, `sha`, performed continual SHA-256 operations, and a low power-consumption workload, `spin`, merely waited while performing no additional work. We executed each workload for approximately the same amount of time, as determined by the system clock. We performed this first investigation on an Intel Core i7-7820HQ CPU running at 2.90 GHz running Linux. We disabled dynamic power/frequency scaling to avoid this affecting our results. We captured power readings from the RAPL interface for comparison purposes. For timing measurements, we captured the number of processor ticks from the performance counter (using the `RDTSC` instruction) and the time difference using the `CLOCK_MONOTONIC` timers. The `CLOCK_MONOTONIC` counter is calibrated by the operating system, using the network Time Protocol (NTP), and therefore provides a reference timebase against which any variation in the processor tick count can be compared. Using the null-hypothesis that "the mean value of a measurement relating to the power consuming task is no different to that of a random selection of measures", we perform t-tests on each measure [Goo+11] and for each task (*cf.* Table 1).

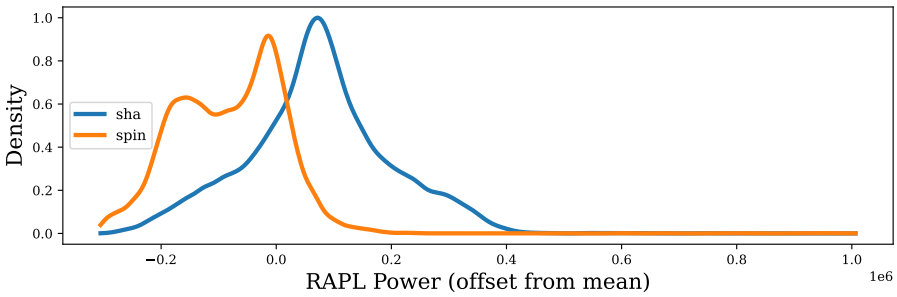


Figure 2: Comparison of activity for RAPL

We see that (unsurprisingly), the RAPL power readings show a very marked difference between the two tasks (*cf.* Figure 2). The leakage of the individual clocks is, as expected, insignificant, but taking the ratio of the two clocks produces $t > 6$, a strong indicator that we should reject the null hypothesis and there is indeed evidence of a correlated leakage (with $p \lesssim 3 \times 10^{-8}$). Plotting a kernel density estimate with a relatively narrow bandwidth (0.1) clearly shows the differences in distribution of the RAPL measurement (Figure 2) and the ratio between `RDTSC` and `CLOCK_MONOTONIC` (Figure 3). The distributions of the single timing measurements are both similar to each other and between the two processing tasks (Figure 18 in Appendix A), confirming that other power-related timing issues (*e.g.* due to DVFS, *cf.* [Wan+22]) are not present.

Following this, we wrote a program that executes different instructions in a tight loop, each executed for a fixed number of times. We again recorded the `CLOCK_MONOTONIC` clocks and the `RDTSC` performance counter and executed the test with successive runs of `NOP`, `ADD` and `IMUL` instructions. We then carried out a t-test between each pair of instructions to

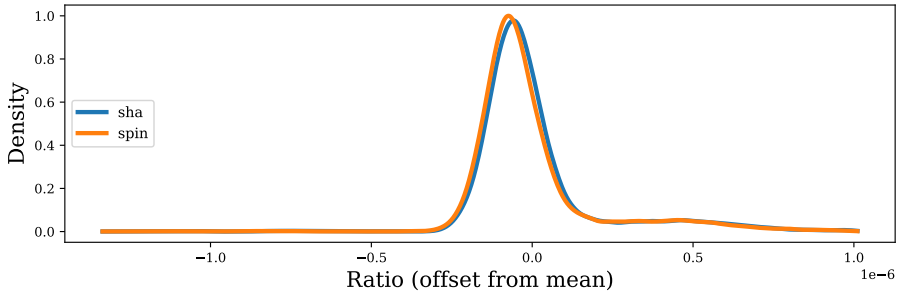


Figure 3: Comparison of activity for `RDTSC/CLOCK_MONOTONIC`

Table 1: Comparison of `spin` and `sha`

Measure	t-value
`RDTSC/CLOCK_MONOTONIC`	-6.44
RAPL power	-163.51
`RDTSC`	-0.19
`CLOCK_MONOTONIC`	-0.15

determine if there was a statistically-significant difference between the means (Table 2). The result demonstrates that individual instructions are distinguishable using the ratio of `RDTSC` and the wall-clock time.

Table 2: Comparison of pairwise t-values for different instructions

Measure	IMUL vs. ADD	NOP vs. ADD	NOP vs. IMUL
`CLOCK_MONOTONIC/RDTSC`	17.87	-22.80	-40.09

Next, we performed an experiment to establish if operands could be distinguished. In this experiment, we executed a fixed length loop of continuous multiplication instructions, with the second operand of the multiplier constant for each run, but selectable for each set of iterations. The initial value of the cumulative multiplication was `3`. We gathered data from two runs, with the multiplier constant set to either `0x00000000` or `0xFFFFFFFF` (Table 3). We captured this data using relatively short timescales (each data point was captured over ∼0.1 sec, approximately 290×10^6 processor ticks).

Although the ratio of the clocks provided the most significant t-values , it was surprising to note that the actual execution time of a series of constant multiplies depends on the operand, which should not be the case according to Intel's documentation[2]. We leave further investigation of this (possible) effect for future work, as it does not directly seem to relate to leakage between clock domains.

From this initial investigation, we concluded that the ratio between different clocks indeed is a source of (minute) power leakage on x86 processors. We attempted to exploit this effect for a CPA but were unsuccessful, although this was not surprising, given the large amount of background noise which is inherent in a system of this sort. However, despite being small, the leakage was statistically very unlikely to have been by chance ($|t| > 18$), and therefore, we opted to investigate the effect on a simpler, embedded processor.

[2] https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/best-practices/data-operand-independent-timing-isa-guidance.html

Table 3: Comparison of multiplication by all-`0` vs. all-`1`

Measure	t-value
`CLOCK_MONOTONIC/RDTSC`	-18.69
`RDTSC`	11.29
`CLOCK_MONOTONIC`	-4.28

2.2 Exploration on i.MX RT1166

An embedded platform provides a more controlled environment for further experiments, in particular, reducing the amount of parallel/background activity in the processor. As we desired a number of different clocking options, we selected a device from the NXP i.MX RT1160 family (specifically, a IMXRT1166DVM5A on an IMXRT1160-EVK evaluation board). This family of devices allows up to six independent timers to be setup, each running from different clock sources. The clock sources range from a high-quality crystal to on-chip PLL and resistor-capacitor (RC) oscillators—in order of quality, from least-to-most-variable. This specific device contains two processor cores, one Cortex-M7 and one Cortex-M4. In all experiments and demonstrations, the Cortex-M7 was used to execute code, and the Cortex-M4 was idle.

Through various experiments on this target, we found a particularly interesting result. We timed a simple table lookup over 1 000 000 loop iterations. Each loop performed a lookup into a customised SBox using random plaintexts, and an implicit key addition of an all zeros key, *cf.* Listing 1. The customised SBox was designed for an even distribution of HWs (*cf.* Listing 2 in Appendix B) in order to reduce the data capture times whilst allowing the effect to be seen clearly.

```
for (uint32_t n = 0; n < iterations; n++) {
    for(int i = 0; i < 4; i++) {
        ct[i] = sbox_even[pt[i]];
    }
}
```

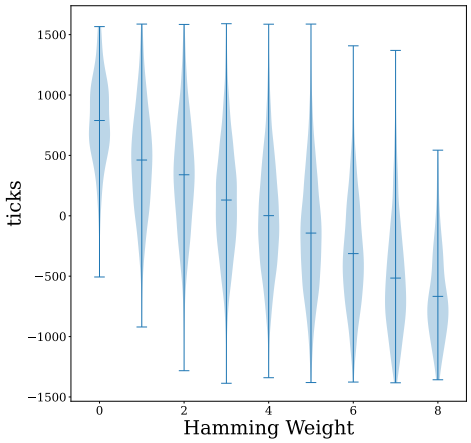
Listing 1: Toy cipher with SBox lookup and (implicit) XOR of all-zero key

We measured the total loop time with a timer clocked directly from a high-quality external crystal oscillator. Such crystals have a maximum deviation over all conditions and lifetime on the order of 50 ppm. We also captured the processor-synchronous `systick` clock. The cycle count of the loop lookup—as measured by `systick`—was expected to be constant, while the variation of the processor PLL clock in relation to the crystal oscillator was expected to vary slightly.

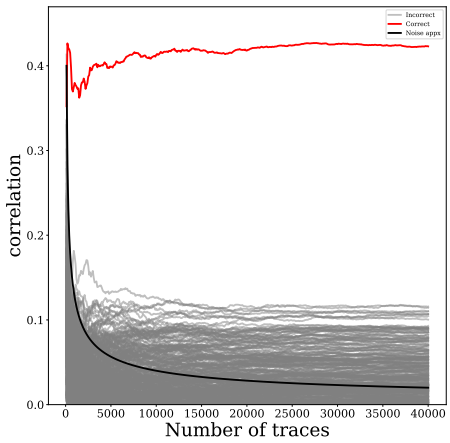
What we actually found was a variation in *both* of the timing measures of approximately 70 ppm, which is *linear* in relation to the HW of the value looked up, *cf.* Figure 5a. Only around 4000 samples are required to produce a correlation value more than 3× that of the incorrect values. To remove the possibility that the memory *location* was the source of leakage, we randomised the SBox content. The leakage remained at a similar level, being clearly visible in a regression analysis (the probability of the line actually having zero slope is $p_{zero} < 1 \times 10^{-27}$ with only 230 samples).

This indicates a potential mechanism for detecting this leakage (which could be employed by either attacker or defender). Timing a loop of a large (but constant) number of `ADD` operations, with the increment being either 0 or non-zero, and the accumulator variable in the asynchronous memory area, should generate an observable difference depending on the increment value.

Experiments with the state of the processor instruction and data caches showed that the data cache being disabled was significant—the data memory was operating asynchronously to the main processor, and therefore reducing the number of hits to the asynchronous bus clearly impacted on the results.



(a) Violin plot showing the linear execution time (with mean subtracted) compared to Hamming Weight (HW) of data



(b) Progression of correlation on toy cipher. Correct key candidate in red, others in gray, noise level in black.

Figure 4: Leakage behaviour on i.MX RT1166 Cortex-M7 processor

Through a further series of experiments, we identified a single instruction which could be changed (from an unnecessary sign-extension to a `NOP`) to reduce the amount of leakage dramatically. We established that disabling the Branch Target Address Cache (BTAC) in the processor reduced the leakage almost completely. We also observed that the silicon temperature had a small effect on the amount of leakage, with the maximum being at normal lab temperatures. Brief experimentation with the core supply voltage (which can be varied under application-software control) indicated that lowering the core voltage reduced the leakage slightly. Raising the core voltage was not attempted to avoid potential damage.

A further intriguing observation is that the "noise floor" of the CPA progression does not follow the theoretical pattern (of $4/\sqrt{n_{samples}}$—shown by the black line in Figure 5b [Osw13]), indicating some non-ideal effects in the leakage. Unfortunately, even tiny changes to the code or location of the code in memory reduced the leakage (as measured by a single timer) by several orders of magnitude, or indeed to zero. Further investigation of these interesting (and potentially significant) effects is left to future work.

Given our demonstration of a correlation between power consumption and measurements of timing, in support of our basic hypothesis that variations in timing can be used to infer power consumption, in the following we describe a number of demonstrative experiments to establish a source of the timing changes. We then revisit the observed leakage on the i.MX RT1166 in that light, and construct software-based leakage primitives and corresponding proof-of-concept attacks (*cf.* Section 5 and Section 6).

3 Simulation-based Experiments

Because the timing variations observed were visible even when measured using the clock synchronous to the processor, we hypothesised that the variation was due to accesses performed to another clock-domain.

We constructed a VHDL simulation to investigate the potential parameters around this leakage (Figure 6). The simulation is a very simple model of two parallel processes

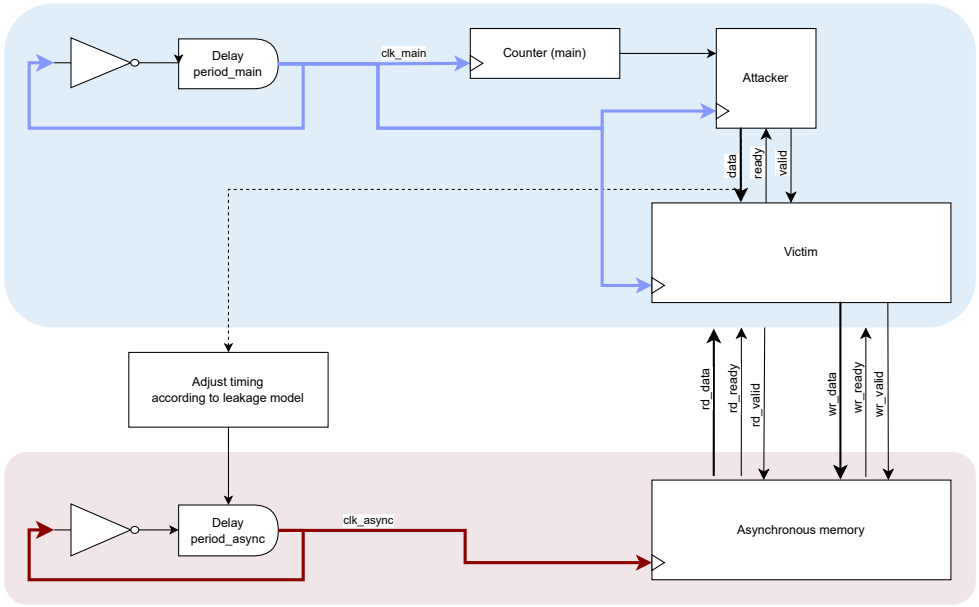


Figure 6: Block diagram of VHDL simulation of leakage due to asynchronous clocks—the two clock domains are highlighted in different colours

('attacker' and 'victim') that communicate using a Advanced eXtensible Interface (AXI)[3]-like pair of "ready/valid" signals. Both these processes are synchronous, driven from the main clock. The simulation was executed with a femtosecond-precision timebase. The attacker process also communicates with an asynchronously-clocked RAM model, using it to read/write an incrementing counter. No clock-domain crossing logic is required, as the simulation model has perfect flip-flops (without any metastable behaviour). This provides an idealised representation of a real-world system. The model allows the asynchronous clock speed of the RAM communication to be manipulated on a cycle-by-cycle basis. This is made dependent on the data being 'processed' in the victim process, provided by the attacker. The victim process runs for a configurable number of clock cycles, before indicating it has 'finished' to the attacker. The attacker measures the 'main' and 'async' clock count differences across this time.

Results In this experiment, we used the clock parameters that were active in the original experiments with the i.MX RT1166. Therefore, the main clock frequency is 600 MHz and the asynchronous clock frequency 198 MHz. The attacker process provides single bits of `0` or `1` to the victim. The victim executes its "processing loop" 3 M times, representing about 5 ms of activity. Based on the data communicated to the victim, the simulation introduces a very small change to the period of the asynchronous clock (of 10 fs). This small change ($\sim$ 2 ppm) results in small changes to the clock count that the victim is measured as being active for. Performing a t-test on the data for 0 and 1 results in a very significant t-value of $>$200 with only 30 samples. The simulated time of this experiment was just 0.16 sec. In this (noise-free) environment, we are able to use the simple cycle-count to confidently distinguish between very small changes in clock frequency with relatively limited data.

[3] https://developer.arm.com/documentation/102202/0300/Channel-transfers-and-transactions

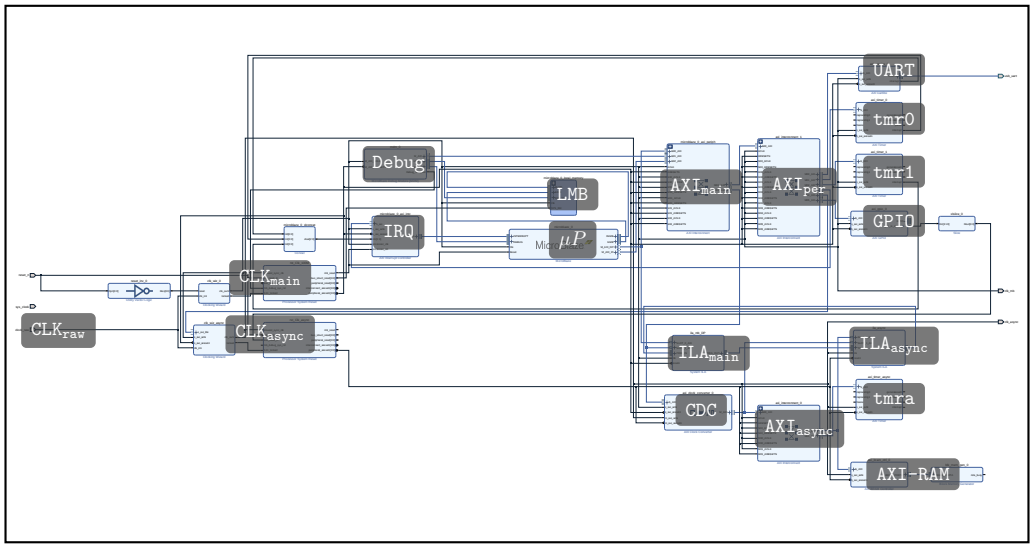


Figure 7: Block diagram of FPGA to explore leakage due to asynchronous clocks

4 Experiments on FPGA

To obtain real-world insights, we next evaluated a target built around a Xilinx Artix-7 FPGA (hosted on a Digilent Arty-7 board). This allows the construction of a controlled lab environment within which we can adjust experimental variables. Again, we only modeled the key parts of the system we are exploring.

In Figure 7, we show the entire system: the main clock is sourced from an external 100 MHz crystal $\texttt{CLK}_{\texttt{raw}}$. This clock is conditioned and divided-down for the majority of the elements using a clock PLL/reset subsystem $\texttt{CLK}_{\texttt{main}}$—this main clock runs at 80 MHz. A second PLL and reset provide a second clock domain $\texttt{CLK}_{\texttt{async}}$. The frequency of this clock is under control of the main microprocessor μP. The microprocessor is tightly coupled (with single-cycle access) to Local Memory Bus (LMB) RAM `LMB`, which is synchronous to the processor clock. The rest of the system is accessed through a synchronous AXI-interconnect $\texttt{AXI}_{\texttt{main}}$, which provides the dividing line between the synchronous peripherals (which form the 'laboratory') and those operating in the asynchronous domain (which are providing the 'experimental' aspects). Two timer peripherals are included in the main microprocessor clock domain `tmr0` and `tmr1`. A General Purpose I/O (GPIO) `GPIO` is provided, which allows the microprocessor to reset the asynchronous clock domain after a change of frequency. In order to make a useful experimental platform, a debug interface `Debug` and Universal Asynchronous Receiver Transmitter (UART) `UART` are present to provide the ability to download/execute code and retrieve results. An interrupt system is also present `IRQ`. These peripherals are connected via an AXI interconnect $\texttt{AXI}_{\texttt{per}}$.

To interface between the main and experimental asynchronous domain, clock-domain crossing (CDC) logic is provided `CDC`. In the asynchronous clock domain, an AXI interconnect $\texttt{AXI}_{\texttt{async}}$ connects a further timer `tmra`, and another RAM block, termed the `AXI-RAM`. To monitor the AXI interconnect signals on either side of the `CDC`, two ILA are present: $\texttt{ILA}_{\texttt{main}}$ and $\texttt{ILA}_{\texttt{async}}$.

We perform an experiment—designed to be similar to the code described in Section 2—such that the core loop (performing the SBox lookups) and the SBox being accessed are both stored in the non-processor-synchronous AXI-RAM. The remainder of the code and data are stored in the fast, processor-synchronous, LMB RAM, allowing the experiment to generate data as quickly as possible.

The frequencies which are available from the on-chip PLL are determined by the driver software, as not all ratios are available. Therefore random frequencies are requested, but the actual frequency of operation is read back from the PLL and recorded. By varying the frequency of the asynchronous RAM, we confirm that the level of leakage is very dependent on the AXI-RAM clock speed (*cf.* Figure 9b).

This dependence happens because—when signals are resampled into the main clock domain from the asynchronous domain—there is a small window around the main clock edge (the so-called 'metastable window') where the capture is unpredictable. During this time, any coincident changes in the signal being sampled will result in stochastic delays at the captured output. This metastable window is dependent on many aspects, including the voltage at the instant of the clock cycle. Therefore a stochastic delay only occurs on the cycles where the two clocks are closely aligned, and is more likely to occur under certain (voltage-related) conditions than others, providing an estimate of power consumption.

If the quotient of the nominal frequencies of the two clocks is rational ($\frac{n}{m}$), the clocks will either align precisely, or be very out of alignment. In either case, the timing of the signals captured from the asynchronous domain is always the same. We expect there to be little dependence on the frequency variation (as the variation is expected to be smaller than the window of sensitivity). There may be a large spread of results if the number of opportunities for this "collision" to occur is large, but the deviations will all be randomly distributed. When the two clocks have no rational relationship, they will move "in and out of phase" with each other and there will be times where the two clocks are very closely, but not exactly, aligned. It is in these instants that unpredictable behaviour related to the power consumption will be captured—Figure 8 shows this behaviour for the non-rational clocks `CLK ref` and `CLK async`.

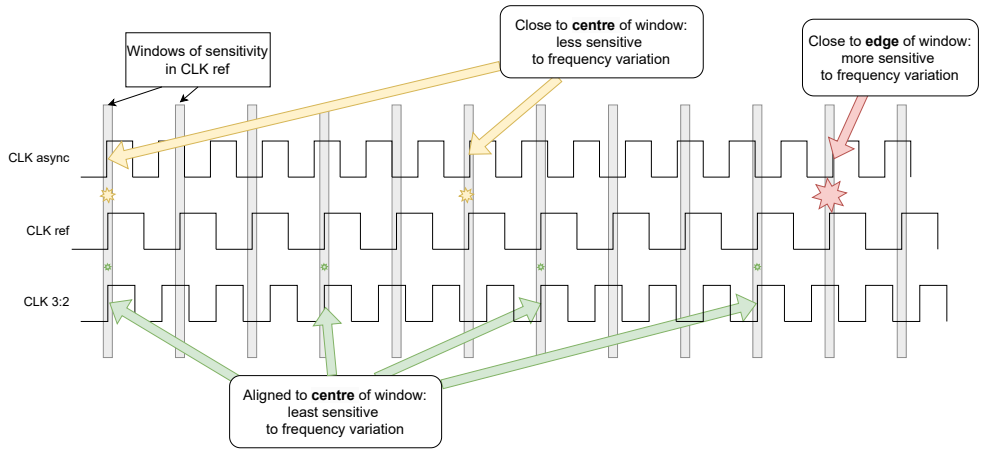


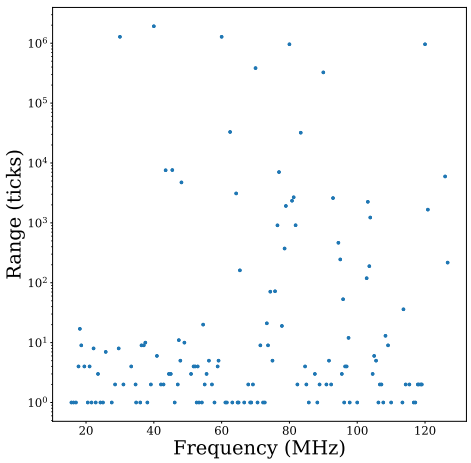
Figure 8: Rational and non-rational divided clocks with respect to a reference clock

We investigated the effect of the clock-frequency of the asynchronous part on the leakage of a series of memory reads. For this experiment, the memory content read was always either `0x00000000` or `0xffffffff`. We measured the time taken to read 100 000 times from RAM using the tick counters `tmr0` and `tmra` and compared t-values for two groups based on the memory read value.

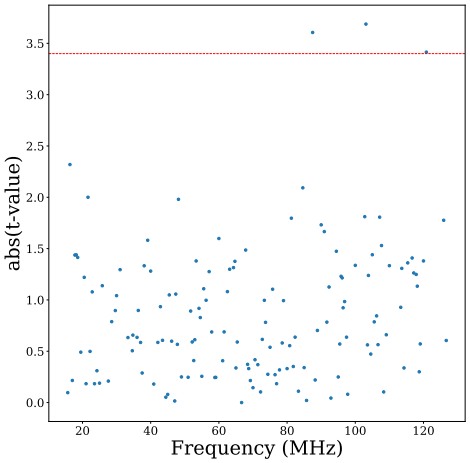
Results Figure 9a shows the variability of the experimental tick count with varying frequency: there is a large distribution of values for the range displayed during the operation, from single cycles, through hundreds, thousands and tens of thousands. We observe that multiples of 10 MHz produce very large (or very small) variations. This fits

our hypothesis, as, depending on the relative phase of the two clocks, these exact multiples of the reference clock would be expected to remain with the same phase relationship, either resulting in "close timing" (causing variation in the tick count) or in "far timing" (causing no clock collisions and hence much smaller variations).

There are numerous frequencies at which there is no variation at all in our (relatively small) experiment, from which we infer that the two clocks do not closely align often enough for there to be a different cycle count. Graphically inspecting the distributions



(a) Variability of the experimental tick count with varying memory clock



(b) Absolute t-values variation with RAM frequency. Red line marks t=3.4 ($p < 1 \times 10^{-4}$)

Figure 9: Experiments on FPGA over different memory clock frequencies

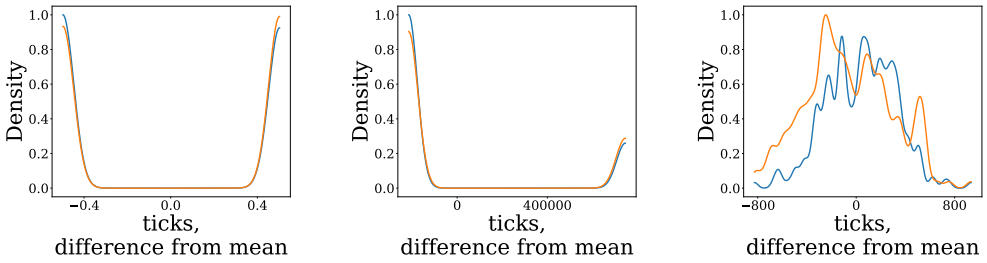
of the sample differences from the mean is interesting: for integer multiples of the main clock frequency, we see a bimodal distribution. The magnitude of the difference across the x-axis is very large at different frequencies. At 100 MHz there is only 1 tick (Figure 10a), whereas at 120 MHz (Figure 10b), the difference is almost 100 000 ticks (or 1 tick per loop iteration). While the total range of values is large, in these two cases, the actual measurement is strongly quantised to the extreme values. The difference in distributions between the large and small HW is still visible on the graphs. In comparison, looking at an asynchronous clock frequency 103.125 MHz, the distribution is less regular (Figure 10c), but the difference between the two distributions for different HWs is visually obvious. Considering the magnitude of the t-values in Figure 9b, we find that for most frequencies, the t-value is below 3.4, indicating no evidence of leakage.

Table 4 shows the three frequencies where the magnitude of t-value for the difference between the two datasets is > 3.4 ($p \ll 0.001$). From this table, we also note that the absolute range of the values is not correlated with the amount of leakage.

Table 4: Comparison of different frequencies on FPGA with significant leakage

Frequency (Hz)	Range (ticks)	t-value (ones vs. zeros)
87500000	3	-3.61
103125000	2247	-3.69
120833336	1664	-3.41

Extending this experiment to perform a toy-cipher SBox lookup (*cf.* Listing 1) and running the asynchronous clock as fast as we are able to, produces results that show a



(a) $f = 100.0\,\text{MHz}$ **(b)** $f = 120.0\,\text{MHz}$ **(c)** $f = 103.125\,\text{MHz}$

Figure 10: Distribution of measurements for HW0 (blue) and HW8 (orange) at different frequencies, highlighting substantial difference at asynchronous clock

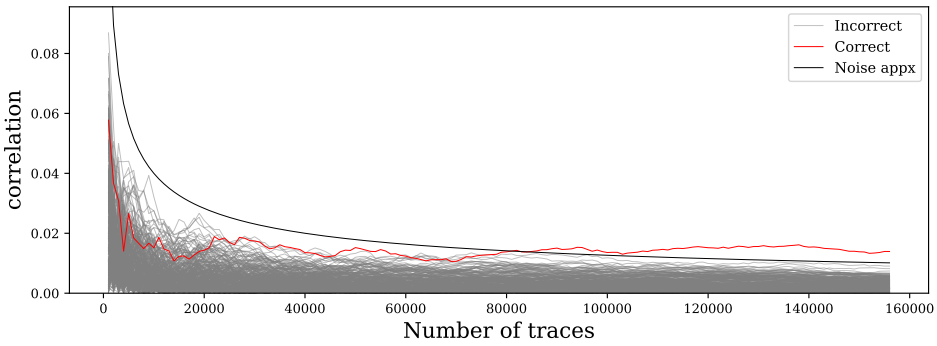


Figure 11: CPA convergence traces for toy-cipher on FPGA platform

leakage related to HW. The linear regression p_{zero} is very low ($p < 4 \times 10^{-8}$), although the r^2 value is also low ($\sim 2 \times 10^{-4}$) due to the level of noise in the system. The slope from the synchronous timer is $\sim$4. This means the variation due to change in HW is 4 ticks per HW bin (out of a total range of $\sim$4700). Still, a CPA correlation is above the theoretical noise value at 1.38 for the correct key candidate, as evident in Figure 11.

5 Clock Variation Measurement over Time

Having investigated the mechanism of the observed leakage due to asynchronous clocks in simulation and on FPGA, we now revisit the i.MX RT1166 behaviour. So far, we have demonstrated a mechanism by which a single sample can be used to measure cumulative energy over some time and infer secrets. In this section, we show how an attacker can gain a trace over time of the i.MX RT1166 power consumption by taking regular samples of a timer during the execution of some process operating on secret data.

More precisely, we perform this capture on the attacker-side of the clock domain boundary by using the Direct Memory Access (DMA) engine within the i.MX SoC. The DMA engine is configured to continually capture the timer value into a RAM buffer. The DMA controller operates in the same clock domain as the RAM block, but in a different clock domain to the timer (and incidentally the processor). Therefore, it is expected to collect the same behaviour, of occasional differences in tick count.

The exact details of the capture are as follows: The DMA engine captures the timer approximately every 26 ticks of the 240 MHz clock (around 108 ns). There is sufficient RAM in the SoC to capture 16 000 samples, over a time period of $\sim$1.73 ms in total.

We initially demonstrate this with a victim process executing two loops of either `NOP`

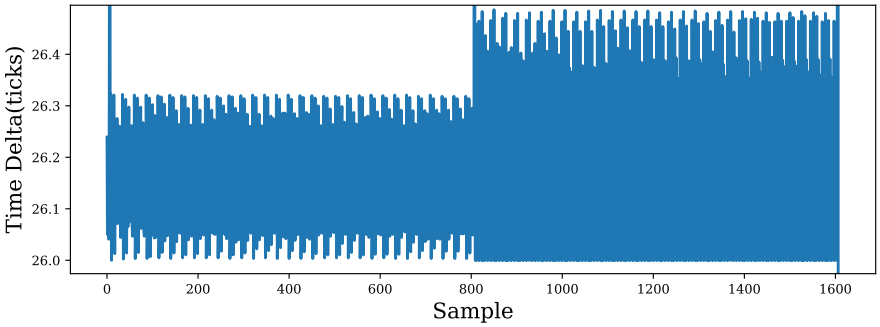


Figure 12: Timer delta capture through DMA over time for `NOP` and `ADD`

or `ADD` instructions. Each loop was tuned to execute in the same amount of time in order to remove an obvious timing side channel. This code executes from the instruction cache and uses the tightly-coupled memory for local variables, meaning it is very well-controlled in time, and operates from a single clock-domain. We take the difference between adjacent time-samples as our data point. A single trace is not useful, as the values are very strongly quantised. Therefore, we capture 1000 such traces and take the pointwise average, with no correction for sample timing. Figure 12 shows a clear transition from the low-power (`NOP`) to the higher-power condition (`ADD`) at (approximately) sample 800.

In order to target a victim process that lasts longer than the available data capture time, we set up the DMA capture to provide a random delay between the start of the victim processing and starting the DMA. This is achieved by starting the DMA from an interrupt, triggered by the same timer as is being used for the sampling. This results in a set of partial traces, each covering a section of the target process execution. The samples read back from the timer directly indicate the (random) DMA delay. Hence, they can be used to reconstruct the full trace based on several sub-traces (recorded at different, random offsets). From the measurements of time, we generate a proxy for power from the difference in consecutive time measurements—corresponding to how long each (notionally constant time) DMA activity took. In our specific example, each DMA took either 26 or 27 cycles.

Due to the random DMA delay, none of these partial time-series are aligned or synchronised; therefore we resample them to a regular grid using a zero-order hold (to extend each sample as a constant until the next sample time). These resampled traces are then averaged to produce a raw power trace. This generates a noisy trace, which benefits from filtering along the time axis. We experimentally evaluated a number of digital filter structures and parameters, which led to the use of a 31-tap digital Bessel-filter[4], with a cutoff frequency (relative to $f_s/2$) of 0.02 and 40 dB of attenuation.

6 Proof-of-Concept Attacks

From this HW-dependent leakage through time measurements, we now build proof-of-concept attacks in a realistic execution environment. Unless otherwise mentioned, for our demonstrations, we extended the i.MX RT1166 software platform to execute the Eclipse ThreadX[5] real-time operating system (on the Cortex-M7 processor as with all previous experiments), showcasing the ability of an attacker thread to exfiltrate secret information from a separate victim thread. We maintain the disabling of the data cache in order to

[4] https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.bessel.html
[5] https://threadx.io/

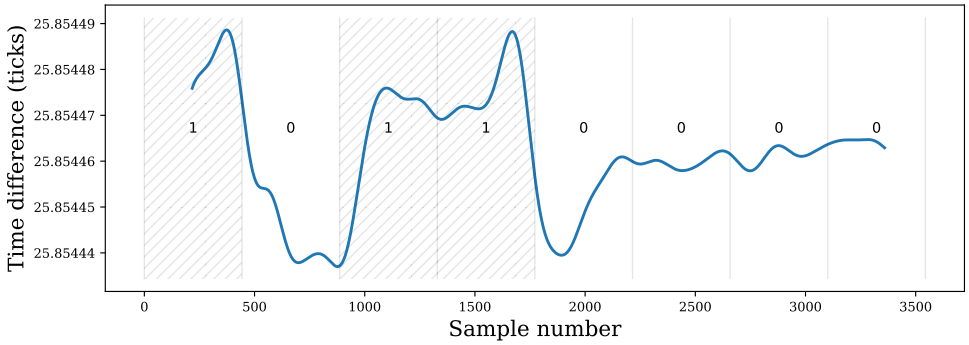


Figure 13: Average of 10 000 interpolated traces after filtering, showing the ability to transmit individual bits through a covert channel.

exclude cache-timing as a leakage path and constrain the variability to only RAM-access timing. Again, no DVFS was active.

6.1 Timing-independent Covert Channel

First, we show how unprivileged access to timer counts can be used to create a (timing-independent) covert channel. The sender process performs an activity which consumes more power to send a `1` bit and idles for the same amount of time (as measured by the processor tick count) to send a `0`. Again, in this example, a series of `ADD` instructions are used for the higher and `NOP` for the lower power conditions, respectively. Using the mechanism described in Section 5, the receiver can capture the data over multiple traces and form a time-series, allowing data to be exfiltrated. In this experiment, an entire example byte (`0b10110000`) can be captured without using any offset at the start of the DMA. The attacker can exfiltrate one byte per set of traces captured. We found that 10 000 traces are necessary for straightforward visual inspection.

This allows one byte every 17.3 s or approximately 58 mBps. Many options likely exist to increase this (low) data rate, but we leave the investigation of the true Signal to Noise Ratio (SNR) and bandwidth of this channel (which would enable calculation of the channel capacity according to [Sha49]) to future work.

6.2 RSA Decryption/Signature

In order to demonstrate the ability to use our side channel to attack longer cryptographic functions, we constructed a simple modular exponentiation following [Knu69, Section 4.6.3]. This uses the mbedTLS v3.6.4[6] bignum functions to perform the multiplications. The victim processes a constant, attacker-provided 'ciphertext'—modelling an RSA decryption/signature operation using the private key as the exponent. This attack was not performed in the context of a ThreadX environment, as the setup only has a single thread of execution at any one time (the attacker starts the DMA and blocks while the victim executes). We do not expect additional noise to be introduced by the Operating System (OS).

For evaluation, we used two 512-bit private keys: Key 0 was randomly generated (`1ebc1cffcb0a683 ... 39e0fa7a6f3ea8b`), while key 1 (`fc3f0fc3f0 ... fc3f0fc3f0`) was selected to be easy to identify in binary form, consisting of a repeating pattern of six `1`s and four `0`s.

[6] https://github.com/Mbed-TLS/mbedtls/releases/tag/v3.6.4

The attacker makes multiple requests to the victim to this operation using one of two keys (unknown to the attacker). To interleave measurements, each time, a key is selected at random by the attacker. Each iteration is also captured with a different random attacker-selected DMA start delay, allowing coverage of the entire execution. The attacker then performs the interpolation and lowpass-filtering described in Section 5.

In this experiment, we set the maximum DMA delay to cover half of the activity, in order to reduce the data-capture time for demonstration. In this case, we captured ∼10 000 traces over approximately 15 h. The entire activity could therefore be captured in 30 h and a realistic 2048 bit key in 5 d (we note that we did not particularly optimise our measurement primitive, and speed-ups could likely be achieved). The measurement primitive induces zero overhead on the victim as the DMA, timer and sample RAM are all operating in a separate clock domain, and on a separate set of bus infrastructure, to the processor on which the victim executes.

The results of this attack are presented in Figure 14. A Savitzky-Golay filter [SG64], with a window size of 500 samples and using a second-order fit, has been additionally applied in order to make the plots more visually accessible. We present the first fifth of the data (corresponding to 56 and 58 bits of the keys, key 0 is the top row and key 1 is the lower row) in the left column. Note that the first 6 bits are obscured by the transient at the start. This transient is caused by the uniform distribution of the DMA delay sampling - this results in this area of the activity being sampled less frequently, resulting in increased noise. Similarly, on the right column, the final fifth of the data (covering 54 bits of each key) are plotted—in this case, the final 8 bits are obscured by a similar transient. It would be straightforward to arrange the DMA delay distribution to provide even coverage of these areas in future work, by starting the DMA some time before the victim activity, and allowing it to continue for some time afterwards, which would result in a small increase in size of the data captured (approximately 2.5 %).

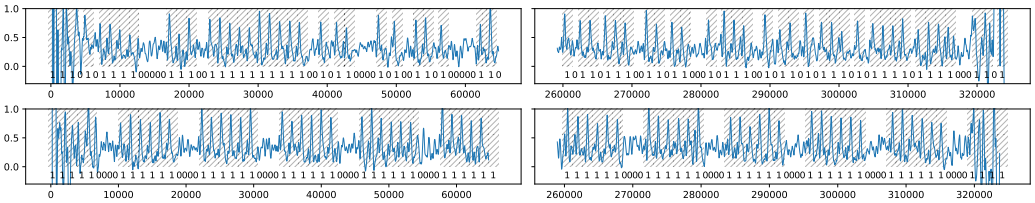


Figure 14: Attack on sections of 512-bit key. The correct key bits are annotated at the bottom of each plot. Top: key 0 (random). Bottom: key 1 (structured). Left: first 55-bits. Right: later 55 bits.

Both the structured and random key bits can be seen to align with the power traces, as would be expected in a traditional SPA-attack, allowing extraction of all but the transiently-obscured 14 bits of the key, leading to a successful attack.

6.3 CPA on TinyAES

In this demonstration, the victim executes TinyAES[7] encryption operations with attacker-controlled plaintext, returning the ciphertext. The attacker loops, incrementing a counter stored in asynchronous RAM, waiting for the victim to finish. In this experiment, the attacker-controlled plaintext, ciphertext, and the background loop counter, as well as the victim's SBox and AES context (which includes the key schedule) are located in asynchronous memory. All other data is located in tightly-coupled RAM, which is clocked by the processor clock. Again, the program code and constants (including the secret key)

[7] https://github.com/kokke/tiny-AES-c

are stored in cached external flash (Figure 15). The two threads are scheduled by the ThreadX operating-system. The measurement does not impact on the victim execution, as it consists only of a timer read before execution and another timer read afterwards.

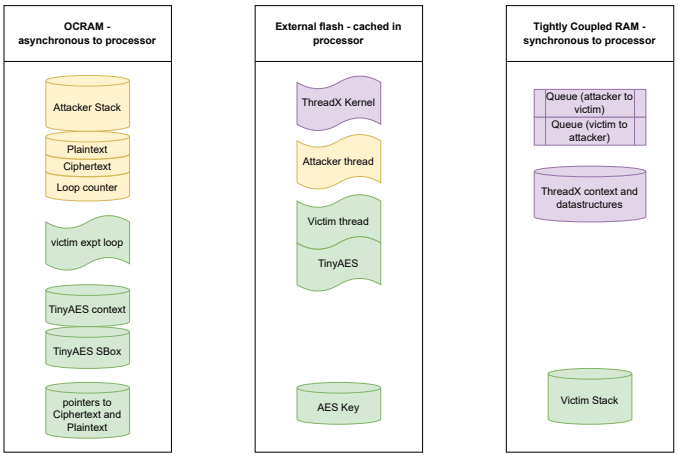


Figure 15: Memory arrangement of TinyAES proof-of-concept on i.MX RT1166

Targeting the first `SubBytes` operation using a HW model, key byte 12 exhibits a clear correlation (exceeding the expected noise level of $4/\sqrt{N}$ by a factor of ∼2.0). We observe that the other key bytes do not leak under this model (Figure 16): we leave the investigation of this single-byte leakage phenomenon for future work.

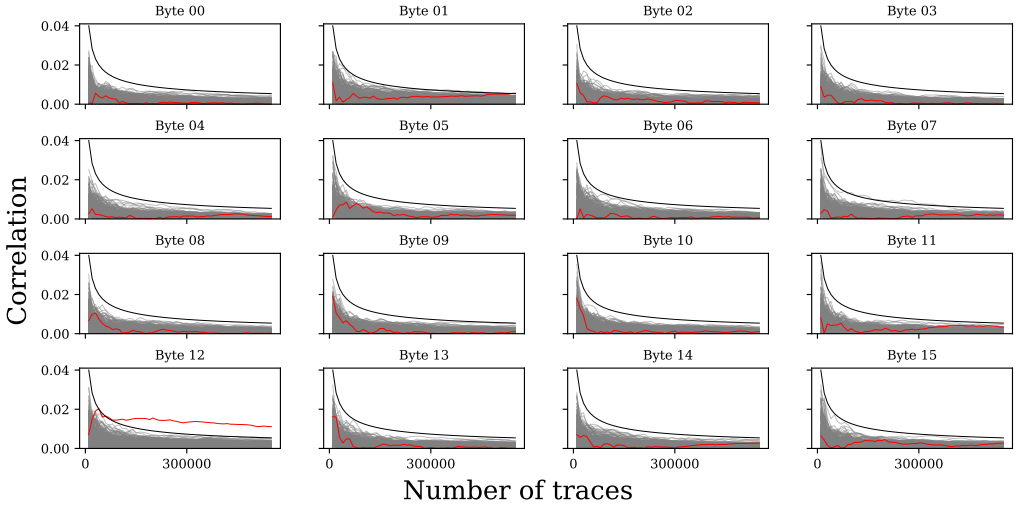


Figure 16: CPA convergence on first `SubBytes` over 545 000 traces on i.MX RT1166

7 Summary

We demonstrated that even carefully-designed constant-time operations can exhibit measurable low-level timing variations, which are dependent on the power being consumed. In addition, continuous fine-grained low-level measurements of time across clock-domains can be used as a proxy for power measurements, revealing details of activity over time which allow the exfiltration of secret information. The details of operation times of non-constant time activities can also be inferred from time-measurements made outside of the privileged domain. Although the effect is very small, it is sufficient to be used to perform traditional power-based side-channel attacks such as CPA and SPA.

7.1 Discussion

The fundamental issue enabling these attacks is the measurement of time from two different sources. These do not have to be formal timers: as we have demonstrated, even the access time of an asynchronous peripheral or RAM block will have timing variability relative to a constant execution time in terms of processor cycles. This timing variability is very small—orders of magnitude smaller than a single clock cycle—but can be effectively measured by exploiting the fact that it will occasionally result in a time measurement which is a whole clock cycle larger. Platforms which are entirely synchronous in nature will not expose such issues. However, these are rare, as interfacing with external devices often requires specific clock frequencies, necessitating the generation of asynchronous clocks.

Based on our understanding of the phenomenon, and the early results presented in Section 2, we believe that this effect is also weakly present on complex x86 processors due to the provision of a real-time clock, which is calibrated with respect to real time, allowing the observation of variations in the processor cycle time. In addition, the external RAM interfaces of such systems are asynchronous. Forcing memory access to external DRAM while secrets are being processed (even from cache) may create a similar side channel.

7.2 Countermeasures

A key mitigation is to locate critical data structures in memory which is synchronous to the processor accessing it. Critical data structures include not just key material, but also lookup tables such as SBoxes and T-tables. Organising these structures in this way removes the power-dependent variability that comes from the asynchronous accesses. We performed such an experiment and demonstrated that no leakage was then detectable from the single timer measurement. However, as demonstrated by the RSA attack, even the behaviour of entirely synchronous code and data can be inferred from the variations in timing of a separate peripheral which is sampled numerous times.

A further approach would be to disable access to all timers. This would be an intolerable imposition on performance-oriented software developers (which is very common in embedded systems work), but perhaps can be considered in deployed products outside of the development process. Quantising the reported time measured by timer peripherals would extend the period over which attacks would have to take place (as the attacker would have to run commensurately more executions of each data point), but does not fix the fundamental problem. In systems which use an external time source to calibrate their idea of time, unprivileged processes could be granted access to the corrected time-base only. Even without access to timers, in a multi-tasking context, an attacker thread which counts loop iterations while the victim is executing can serve as a time base [Lip+16].

All usual mitigations against power side-channels remain applicable, *e.g.* masking for symmetric cryptography [RP10] and blinding [Koc96] in the case of asymmetric cryptography. Interestingly, mitigations which introduce jitter are unlikely to be successful, as our sampling is very slow in relation to the process being attacked and therefore the jitter

would have to be greater than the time between samples. In the case of the DMA-based attacks, this is on the order of 100 ns. When a single data point is used to perform the attack (as in our AES demonstration), any jitter will still result in a leakage captured within that single sample time.

Acknowledgements

Martin gratefully acknowledges the support of his employer, ZF Automotive UK Ltd, and the scholarships provided by the Universities of Birmingham and Durham. This research was partially funded by the Engineering and Physical Sciences Research Council (EPSRC) under grants EP/X03738X/1 and EP/X03738X/2.

References

[Abr02] D. Abramovitch. "Phase-locked loops: a control centric tutorial". In: *Proceedings of the 2002 American Control Conference (IEEE Cat. No.CH37301)*. 2002. DOI: 10.1109/ACC.2002.1024769.

[Ayo+24] Pierre Ayoub et al. "PhaseSCA: Exploiting Phase-Modulated Emanations in Side Channels". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2025.1 (Dec. 2024), pp. 392–419. DOI: 10.46586/tches.v2025.i1.392-419. URL: https://tches.iacr.org/index.php/TCHES/article/view/11934.

[Ban17] Dean Banerjee. *PLL Performance, Simulation, and Design, 5th Edition (SNAA106c)*. 2017. URL: https://www.ti.com/lit/ml/snaa106c/snaa106c.pdf.

[BB03] David Brumley and Dan Boneh. "Remote Timing Attacks Are Practical". In: *USENIX Security 2003*. USENIX Association, Aug. 2003.

[BCO04] Éric Brier, Christophe Clavier, and Francis Olivier. "Correlation Power Analysis with a Leakage Model". In: *CHES*. 2004.

[Ber05] Daniel J Bernstein. *Cache-timing attacks on AES*. 2005. URL: https://cr.yp.to/antiforgery/cachetiming-20050414.pdf.

[BM06] Joseph Bonneau and Ilya Mironov. "Cache-Collision Timing Attacks Against AES". In: *Cryptographic Hardware and Embedded Systems - CHES 2006*. Ed. by Louis Goubin and Mitsuru Matsui. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 201–215.

[Dog+11] Julien Doget et al. "Univariate side channel attacks and leakage modeling". In: *J. Cryptogr. Eng.* 1.2 (2011), pp. 123–144. DOI: 10.1007/S13389-011-0010-2. URL: https://doi.org/10.1007/s13389-011-0010-2.

[FBL24] Anis Fellah-Touta, Lilian Bossuet, and Carlos Andres Lara-Nino. "A Lightweight Non-Oscillatory Delay-Sensor for Remote Power Analysis". In: *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 2024, pp. 343–348. DOI: 10.1109/HOST55342.2024.10545353.

[Gie+08] Benedikt Gierlichs et al. "Mutual Information Analysis A Generic Side-Channel Distinguisher". In: *Lecture Notes in Computer Science* 5154 (2008), pp. 426–442. URL: https://api.semanticscholar.org/CorpusID:3754674.

[GKT19] Dennis R. E. Gnad, Jonas Krautter, and Mehdi B. Tahoori. "Leaky Noise: New Side-Channel Attack Vectors in Mixed-Signal IoT Devices". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2019.3 (2019), pp. 305–339. DOI: 10.13154/tches.v2019.i3.305-339. URL: https://tches.iacr.org/index.php/TCHES/article/view/8297.

[GMO01] Karine Gandolfi, Christophe Mourtel, and Francis Olivier. "Electromagnetic Analysis: Concrete Results". In: *Cryptographic Hardware and Embedded Systems — CHES 2001*. Ed. by Çetin K. Koç, David Naccache, and Christof Paar. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 251–261. ISBN: 978-3-540-44709-2.

[Goo+11] Gilbert Goodwill et al. *A testing methodology for side-channel resistance validation*. 2011. URL: https://csrc.nist.gov/csrc/media/events/non-invasive-attack-testing-workshop/documents/08_goodwill.pdf (visited on 03/2022).

[Gra+21] Joseph Gravellier et al. "SideLine: How Delay-Lines (May) Leak Secrets from Your SoC". In: *COSADE 2021*. Ed. by Shivam Bhasin and Fabrizio De Santis. Vol. 12910. LNCS. Springer, Cham, Oct. 2021, pp. 3–30. DOI: 10.1007/978-3-030-89915-8_1.

[Kel+98] John Kelsey et al. "Side channel cryptanalysis of product ciphers". In: *Computer Security — ESORICS 98*. Ed. by Jean-Jacques Quisquater et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 97–110.

[Kim+19] Jaehun Kim et al. "Make Some Noise. Unleashing the Power of Convolutional Neural Networks for Profiled Side-channel Analysis". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 3 (2019). DOI: 10.13154/tches.v2019.i3.148-179. URL: https://tches.iacr.org/index.php/TCHES/article/view/8292.

[KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. "Differential Power Analysis". In: *Advances in Cryptology — CRYPTO' 99*. Ed. by Michael Wiener. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 388–397. ISBN: 978-3-540-48405-9.

[Knu69] Donald E. Knuth. *The Art of Computer Programming: Vol. 2 Seminumerical Algorithms*. Addison-Wesley Series in Computer Science and Information Processing. Addison-Wesley, 1969.

[Koc+19] Paul Kocher et al. "Spectre Attacks: Exploiting Speculative Execution". In: *Proceedings of the 40th IEEE Symposium on Security and Privacy*. 2019, pp. 1–19. DOI: 10.1109/SP.2019.00001.

[Koc96] Paul C. Kocher. "Timing Attacks on Implementations of Diffie-Hellman RSA, DSS, and Other Systems". In: *Advances in Cryptology — CRYPTO '96*. Ed. by Neal Koblitz. 1996. URL: https://link.springer.com/chapter/10.1007%2F3-540-68697-5_9#aboutcontent.

[Lip+16] Moritz Lipp et al. "ARMageddon: Cache Attacks on Mobile Devices". In: *25th USENIX Security Symposium (USENIX Security 16)*. Austin, TX: USENIX Association, Aug. 2016, pp. 549–564. ISBN: 978-1-931971-32-4. URL: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/lipp.

[Lip+18] Moritz Lipp et al. "Meltdown: Reading Kernel Memory from User Space". In: *Proceedings of the 27th USENIX Security Symposium*. 2018, pp. 973–990. DOI: 10.1177/1555343419850132.

[Lip+21] Moritz Lipp et al. “PLATYPUS: Software-based Power Side-Channel Attacks on x86”. In: *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021, pp. 355–371. DOI: 10.1109/SP40001.2021.00063.

[Lon+15] Jake Longo et al. “SoC It to EM: ElectroMagnetic Side-Channel Attacks on a Complex System-on-Chip”. In: *Cryptographic Hardware and Embedded Systems - CHES 2015 - 17th International Workshop, Saint-Malo, France, September 13-16, 2015, Proceedings*. Ed. by Tim Güneysu and Helena Handschuh. Vol. 9293. Lecture Notes in Computer Science. Springer, 2015, pp. 620–640. DOI: 10.1007/978-3-662-48324-4_31. URL: https://doi.org/10.1007/978-3-662-48324-4%5C_31.

[MA0-] Michael McKerns and Michael Aivazis. *pathos: a framework for heterogeneous computing*. 2010-. URL: https://uqfoundation.github.io/project/pathos.

[Max73] James Clerk Maxwell. *A Treatise on Electricity and Magnetism*. Oxford University Press, 1873.

[McK+11] M.M. McKerns et al. “Building a framework for predictive science”. In: *Proceedings of the 10th Python in Science Conference*. 2011. URL: https://eprint.iacr.org/1202.1056.

[MOP10] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. 1st. Springer Publishing Company, Incorporated, 2010. ISBN: 1441940391.

[OD19] Colin O’Flynn and Alex Dewar. “On-Device Power Analysis Across Hardware Security Domains.: Stop Hitting Yourself.” In: 2019 (2019). DOI: 10.13154/tches.v2019.i4.126-153. URL: https://tches.iacr.org/index.php/TCHES/article/view/8347.

[OFl24] Colin O’Flynn. *Phase Modulation Side Channels: Jittery JTAG for On-Chip Voltage Measurements*. Cryptology ePrint Archive, Paper 2024/1107. 2024. URL: https://eprint.iacr.org/2024/1107.

[Ohm27] G.S. Ohm. *Die galvanische Kette, mathematisch bearbeitet*. Reimann, 1827.

[Osw13] David Oswald. “Implementation Attacks: From Theory to Practice”. PhD thesis. Ruhr-University Bochum, 2013. URL: http://www.emsec.rub.de/media/crypto/veroeffentlichungen/2010/09/07/paper.pdf.

[Per05] Colin Percival. *CACHE MISSING FOR FUN AND PROFIT*. 2005. URL: https://www.daemonology.net/papers/htt.pdf.

[RP10] Matthieu Rivain and Emmanuel Prouff. “Provably Secure Higher-Order Masking of AES”. In: *Cryptographic Hardware and Embedded Systems, CHES 2010*. Ed. by Stefan Mangard and François-Xavier Standaert. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 413–427. ISBN: 978-3-642-15031-9.

[Sch+23] Kai Schoos et al. “JitSCA: Jitter-based Side-Channel Analysis in Picoscale Resolution”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2023.3 (June 2023), pp. 294–320.

[SG64] Abraham. Savitzky and M. J. E. Golay. “Smoothing and Differentiation of Data by Simplified Least Squares Procedures.” In: *Analytical Chemistry* 36.8 (1964), pp. 1627–1639. DOI: 10.1021/ac60214a047. eprint: https://doi.org/10.1021/ac60214a047. URL: https://doi.org/10.1021/ac60214a047.

[Sha49] C.E. Shannon. “Communication in the Presence of Noise”. In: *Proceedings of the IRE* 37.1 (1949), pp. 10–21. DOI: 10.1109/JRPROC.1949.232969.

[SLP05] Werner Schindler, Kerstin Lemke, and Christof Paar. "A Stochastic Model for Differential Side Channel Cryptanalysis". In: *CHES 2005*. Ed. by Josyula R. Rao and Berk Sunar. Vol. 3659. LNCS. Springer, Berlin, Heidelberg, Aug. 2005, pp. 30–46. DOI: 10.1007/11545262_3.

[SM22] Tobias Schneider and Amir Moradi. "Leakage Assessment Methodology". In: (2022), pp. 495–513. DOI: 10.1007/978-3-662-48324-4_25. URL: https://doi.org/10.1007/978-3-662-48324-4_25.

[Udu+21] Brian Udugama et al. "VITI: A Tiny Self-Calibrating Sensor for Power-Variation Measurement in FPGAs". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2022.1 (Nov. 2021), pp. 657–678. DOI: 10.46586/tches.v2022.i1.657-678. URL: https://tches.iacr.org/index.php/TCHES/article/view/9311.

[Udv+21] Balazs Udvarhelyi et al. "On the Security of Off-the-Shelf Microcontrollers: Hardware Is Not Enough". In: *Smart Card Research and Advanced Applications*. Ed. by Pierre-Yvan Liardet and Nele Mentens. Cham: Springer International Publishing, 2021, pp. 103–118. ISBN: 978-3-030-68487-7.

[VMC19] Aurélien Vasselle, Philippe Maurine, and Maxime Cozzi. "Breaking Mobile Firmware Encryption through Near-Field Side-Channel Analysis". In: *Proceedings of the 3rd ACM Workshop on Attacks and Solutions in Hardware Security Workshop* (2019).

[Wan+22] Yingchen Wang et al. "Hertzbleed: Turning Power Side-Channel Attacks Into Remote Timing Attacks on x86". In: *Proceedings of the USENIX Security Symposium (USENIX)*. 2022.

A Distribution for Different Measurements on x86

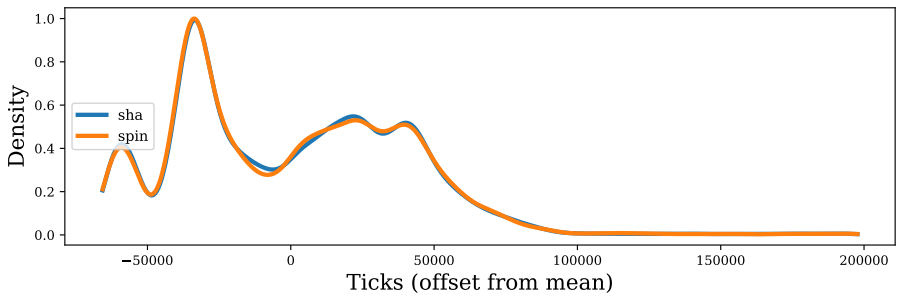


Figure 17: Comparison of activity for `CLOCK_MONOTONIC`

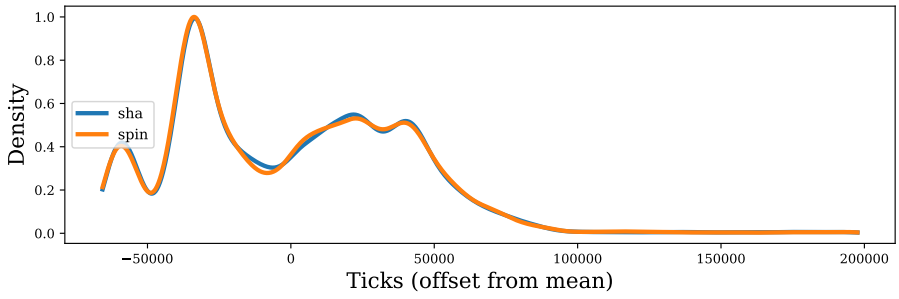


Figure 18: Comparison of activity for `RDTSC`

B Appendix - "sbox_even"

```
#define SBOX_EVEN_LENGTH (252u)
const uint8_t sbox_even[SBOX_EVEN_LENGTH] =
{0xff, 0x44, 0x37, 0x0, 0xf0, 0x2, 0x6f, 0x0, 0xa0, 0x10, 0xfc, 0x8d, 0xff, 0x9f, 0x0,
0xda, 0xff, 0x8e, 0xba, 0x0, 0x41, 0x0, 0xef, 0x8, 0xd2, 0xff, 0x0, 0xd1, 0xbf, 0x8,
0xcf, 0x8f, 0x4f, 0xac, 0x94, 0xfd, 0xff, 0x0, 0x0, 0xdb, 0x0, 0x1, 0xff, 0xfa, 0xd,
0x12, 0xff, 0x8f, 0x2d, 0x0, 0x0, 0xff, 0xef, 0x80, 0x0, 0x64, 0xdf, 0x41, 0x90, 0x4,
0xaf, 0xf3, 0x60, 0x80, 0xed, 0x40, 0x57, 0x52, 0x3b, 0xc3, 0xd8, 0x39, 0x2, 0x22,
0xfb, 0x0, 0x0, 0x54, 0xbb, 0x40, 0xd9, 0x5f, 0xc3, 0x0, 0x60, 0xfb, 0x40, 0x6e,
0x6b, 0x20,0x99, 0x0, 0xff, 0xf8, 0x6c, 0xb0, 0x80, 0x12, 0x9f, 0xbf, 0xff, 0xef, 0x7,
0x0, 0xff, 0xf1, 0x2, 0x39, 0x9f, 0xff, 0x1, 0x0, 0x60, 0x88, 0x43, 0x4e, 0x0, 0x4c,
0x4b, 0x6b,0xf0, 0x7f, 0x8, 0x0, 0xeb, 0xb4, 0xde, 0xff, 0xff, 0x26, 0xdb, 0x8a, 0x83,
0x0, 0xde, 0x13, 0x10, 0x1, 0xff, 0x64, 0xfb, 0xd5, 0x45, 0x0,  0x6a, 0x80, 0xfe, 0x90,
0xbf, 0x20,0x33, 0xff, 0xff, 0x0, 0xff, 0x9e, 0x7f, 0xf7, 0x2, 0xba, 0x7f, 0x5e, 0xb,
0x90, 0xcf, 0x9f, 0x34, 0x98, 0x38, 0xff, 0xff, 0xfe, 0xee, 0xa9, 0xd4, 0x92, 0xef,
0x4, 0x0, 0x44, 0x79, 0x0, 0x9, 0xaf, 0x63, 0xcf, 0x5b, 0x0, 0xa, 0xd3, 0x3f, 0xd5,
0xef, 0x18, 0xff, 0x6f, 0x48, 0xce, 0xff, 0xc, 0xfe, 0xb8, 0xfe, 0x32, 0x12, 0x20,
0xff, 0x77, 0x21, 0xf7, 0x8, 0x2, 0x48, 0x0, 0xff, 0xff, 0xef, 0x0, 0x5, 0xa4, 0x79,
0x30, 0xff, 0xfe, 0xb0, 0x73, 0x33, 0x24, 0x20, 0xf3, 0x2b, 0x31, 0x13, 0xdc, 0x89,
0xff, 0x99, 0x29, 0x77, 0xb2, 0xc, 0xb7, 0x98, 0x3b, 0xdf, 0xff, 0xfd, 0x81, 0xbf,
0xf7, 0x6e, 0x2};
```

Listing 2: Custom SBox with even distribution of HW